REȚELE DE CALCULATOARE · SEMESTRUL 1 · CURSUL 3

Nivelul aplicație: socket-uri, Web-ul și HTTP/1.1

Pachetul didactic al săptămânii 3: note de curs, seminar, laborator Linux și laborator Windows (câte două ore fiecare)

Conf.univ.dr. Popescu-Bodorin Nicolaie, 2026
+40722882471, bodorin(on)live.com, https://lmrec.org/bodorin/

Convenție de terminologie
La prima apariție în fiecare parte a documentului, termenii de specialitate traduși în română sunt urmați, în paranteză, de termenul consacrat în limba engleză sau de forma „romgleză” folosită curent de practicieni — de exemplu, **comutator de pachete** (*packet switch*), **memorie tampon** (*buffer*, în romgleză „buffer”). Comenzile, numele câmpurilor din Wireshark și rezultatele afișate de programe sunt lăsate în engleză, așa cum apar pe ecran.

Partea	Conținut	Durată
A	Cursul 3 — Nivelul aplicație: socket-uri, Web-ul și HTTP/1.1	2 h
B	Seminarul 3 — Timpi HTTP, mesaje și fluxuri de octeți	2 h
C	Laboratorul 3 (Linux) — Socket-uri I: un client și un server TCP în C	2 h
D	Laboratorul 3 (Windows) — Socket-uri I: un client și un server TCP cu Winsock	2 h

Partea A · Cursul 3

După acest curs ar trebui să puteți
• să comparați arhitecturile de aplicații client–server și de la egal la egal (peer-to-peer);
• să explicați cum este adresat un proces în Internet și ce este un socket;
• să spuneți de ce poate avea nevoie o aplicație de la nivelul transport și să alegeți între TCP și UDP;
• să descrieți elementele Web-ului: obiecte, URL-uri și schimbul cerere–răspuns HTTP;
• să calculați timpii de încărcare a unei pagini pentru conexiuni HTTP nepersistente și persistente;
• să citiți și să scrieți mesaje HTTP/1.1 și să spuneți cum află receptorul unde se termină un mesaj;
• să enumerați apelurile de sistem pe care le fac un client TCP și un server TCP, și în ce ordine.

Planul celor două ore

Partea	Subiect	Timp
Ora 1		
1	De la baza stivei spre vârf	5 min

Partea	Subiect	Timp
2	Arhitecturi de aplicații	10 min
3	Procese, adrese și socket-uri	15 min
4	De ce au nevoie aplicațiile de la nivelul transport	15 min
5	Ce definește un protocol de nivel aplicație	5 min
	Pauză	10 min
Ora 2		
6	Web-ul: obiecte și URL-uri	8 min
7	Conexiunile HTTP și costul lor în timpi dus-întors	15 min
8	Mesajele HTTP/1.1, metodele și codurile de stare	17 min
9	Interfața socket, pas cu pas; rezumat	10 min

1. De la baza stivei spre vârf

În primele două săptămâni am privit Internetul ca întreg: părțile lui, nivelurile și performanța. De azi coborâm prin stivă (*stack*) câte un nivel o dată, pornind de sus — de la **nivelul aplicație** (*application layer*), nivelul ale cărui protocoale le folosiți zilnic și cel pentru care cei mai mulți dintre voi veți programa tot restul carierei. Săptămâna aceasta se întâmplă două lucruri. La curs studiem cum sunt organizate aplicațiile de rețea și protocolul Web-ului, HTTP. La laborator scrieți primele programe de rețea, un client TCP și un server TCP, folosind aceeași interfață pe care o folosește orice browser și orice server web.

2. Arhitecturi de aplicații

O aplicație de rețea este un ansamblu de programe care rulează pe sisteme terminale (*end systems*) diferite și comunică prin rețea. Proiectantul ei alege mai întâi o **arhitectură** (*architecture*) — modul în care sunt împărțite rolurile între programe. Ruterele și switch-urile nu sunt implicate: ele nu rulează cod de aplicație, motiv pentru care aplicații noi pot fi lansate fără nicio schimbare în rețea.

	Client–server	De la egal la egal (peer-to-peer, P2P)
Roluri	Un **server** mereu pornit, cu o adresă fixă și cunoscută, deservește mulți **clienți**. Clienții nu comunică între ei.	Gazde conectate intermitent (**entități pereche**, *peers*) comunică direct una cu alta; fiecare este și client, și server.
Exemple	Web, e-mail, servicii bancare online, majoritatea serviciilor cloud	BitTorrent, părți ale unor sisteme de apeluri video și de actualizare software
Scalare	Furnizorul serviciului trebuie să cumpere capacitate pentru toți clienții; serviciile mari folosesc **centre de date** (*data centres*) cu multe servere în spatele unui singur nume.	**Auto-scalare** (*self-scaling*): fiecare entitate nouă aduce cerere *și* capacitate.
Dificultăți	Costul serverelor și al lățimii de bandă; un punct unic de control și de cădere (*single point of failure*).	Entitățile vin și pleacă, stau în spatele NAT (*network address translation*), au legături de acces asimetrice; securitatea și stimulentele sunt dificile.

Multe sisteme reale sunt **hibride** (*hybrid*): un serviciu de mesagerie folosește servere ca să găsească utilizatorii și să rețină mesajele pentru cei deconectați, în timp ce un apel vocal sau video poate circula direct între cele două telefoane.

3. Procese, adrese și socket-uri

Nu gazdele comunică, ci **procesele** (*processes*) — programe care rulează pe gazde. Două procese de pe aceeași gazdă comunică prin sistemul de operare (*pipes*, memorie partajată). Procesele de pe gazde diferite comunică schimbând **mesaje** (*messages*) prin rețea. În fiecare schimb, procesul care îl inițiază este **clientul**, iar cel care așteaptă să fie contactat este **serverul**. (Într-un transfer P2P de fișiere, aceeași entitate este client într-un schimb și server în altul.)

3.1 Socket-ul: o ușă între aplicație și transport

Un proces trimite și primește mesaje printr-o interfață software numită **socket** (în română, rar, „soclu”; în practică „socket”). O imagine bună este o ușă: procesul își împinge mesajul afară pe ușa lui și se bazează pe infrastructura de transport de dincolo ca să îl ducă la ușa procesului destinatar. Dezvoltatorul aplicației controlează tot ce se află de partea aplicației a socket-ului; de partea transportului, singurele alegeri sunt protocolul de transport și câțiva parametri (dimensiunile memoriilor tampon — *buffers*, temporizările — *timeouts*). Secțiunea 9 și laboratorul arată în detaliu apelurile socket.

3.2 Adresarea unui proces

Ca să trimită un mesaj unui proces de pe altă gazdă, un expeditor are nevoie de două lucruri:

- **adresa IP** (*IP address*) a gazdei (32 de biți în IPv4, 128 de biți în IPv6), care identifică interfața de rețea a gazdei; și
- un **număr de port** (*port number*, 16 biți), care identifică procesul destinatar — strict vorbind, socket-ul destinatar — pe acea gazdă.

Serviciile bine-cunoscute folosesc porturi bine-cunoscute (*well-known ports*): HTTP 80, HTTPS 443, DNS 53, SMTP 25, SSH 22. Sistemul de operare al unui client îi dă socket-ului un **port efemer** (*ephemeral port*), temporar. O conexiune TCP este identificată de patru numere împreună, **cvadruplul** (*4-tuple*) *(IP sursă, port sursă, IP destinație, port destinație)*. Așa deosebește un server web cu mii de conexiuni deschise către portul 443 conexiunile între ele: nu există două cu același cvadruplu.

4. De ce au nevoie aplicațiile de la nivelul transport

Când creați un socket, alegeți un protocol de transport. Alegerea potrivită depinde de nevoile aplicației, pe patru dimensiuni:

- **Transfer fiabil de date** (*reliable data transfer*). Trebuie să ajungă fiecare octet, intact și în ordine? Un transfer de fișiere sau o tranzacție bancară nu tolerează nicio pierdere. Un apel video tolerează câteva: un fragment de cadru pierdut este mai bun decât unul întârziat.
- **Debit** (*throughput*). Are aplicația nevoie de o rată minimă (un flux video la 5 Mbit/s) sau poate folosi cât este disponibil (e-mail, transfer de fișiere — aplicații *elastice*, *elastic applications*)?

- **Temporizare** (*timing*). Trebuie datele să ajungă într-un anumit interval? Jocurile interactive și vocea au nevoie de zeci de milisecunde; unei descărcări de fișier nu îi pasă.
- **Securitate** (*security*). Confidențialitatea, integritatea și autentificarea capetelor.

Aplicație	Pierdere de date	Debit	Sensibilă la timp
Transfer de fișiere, descărcare	fără pierderi	elastic	nu
E-mail	fără pierderi	elastic	nu
Pagini web	fără pierderi	elastic	oarecum (secunde)
Apeluri vocale și video	tolerează pierderi	audio: kbit/s; video: Mbit/s	da, ~100 ms
Video înregistrat, în flux (*streaming*)	tolerează pierderi	Mbit/s	da, câteva secunde
Jocuri interactive	tolerează pierderi	de la kbit/s în sus	da, zeci de ms
Mesagerie text	fără pierderi	elastic	și da, și nu

4.1 Ce oferă Internetul

	TCP	UDP
Conexiune	**orientat pe conexiune** (*connection-oriented*): o stabilire a conexiunii (*handshake*) înainte ca datele să circule	fără conexiune (*connectionless*): fiecare datagramă este de sine stătătoare
Fiabilitate	**flux de octeți fiabil și ordonat** (*reliable, in-order byte stream*)	deloc: datagramele pot fi pierdute, duplicate, reordonate
Controlul fluxului (*flow control*)	da: nu copleșește niciodată receptorul	nu
Controlul congestiei (*congestion control*)	da: încetinește când rețeaua este congestionată	nu
Granițele mesajelor (*message boundaries*)	**niciuna** — un flux de octeți	păstrate: un send = o datagramă
Garanții de timp și de debit	niciuna	niciuna
Utilizatori tipici	HTTP/1.1 și HTTP/2, SMTP, SSH, transfer de fișiere	DNS, voce și video, jocuri; QUIC (și deci HTTP/3)

Niciunul dintre protocoale nu garantează temporizarea sau debitul: Internetul este fără garanții (*best effort*), cum am văzut în Cursul 1. Aplicațiile care au nevoie de ele se descurcă prin proiectare — memorare tampon, adaptarea ratei, alegerea unor servere apropiate. Niciunul dintre protocoale nu criptează. Securitatea se adaugă deasupra transportului prin **TLS** (*Transport Layer Security*), o bibliotecă pe care aplicația o apelează în locul socket-ului simplu și care oferă o conexiune TCP criptată și autentificată; HTTPS este HTTP peste TLS. QUIC, protocolul de transport de sub HTTP/3, rulează peste UDP și reunește într-un singur protocol fiabilitatea, controlul congestiei și criptarea TLS.

> **De ce ar alege cineva UDP?**
> Pentru că uneori serviciile TCP stau în cale. Retransmisiile și ordonarea TCP pot reține date proaspete în spatele unor date vechi pierdute; stabilirea conexiunii costă un timp dus-întors înaintea primului octet; controlul congestiei poate încetini un expeditor care ar prefera să piardă ceva date decât să ajungă târziu. O aplicație peste UDP poate adăuga exact fiabilitatea de care are nevoie — sau poate construi un transport cu totul nou, cum face QUIC.

5. Ce definește un protocol de nivel aplicație

Un protocol de nivel aplicație (*application-layer protocol*) fixează, pentru procesele unei aplicații:

- **tipurile de mesaje** (*message types*) schimbate — de exemplu cereri și răspunsuri;
- **sintaxa** (*syntax*) fiecărui tip — câmpurile lui și modul în care sunt delimitate;
- **semantica** (*semantics*) câmpurilor — ce înseamnă informația din ele;
- **regulile** (*rules*) după care un proces trimite mesaje și răspunde la ele.

Unele protocoale sunt **deschise** (*open*), specificate în RFC-uri publice, astfel încât oricine poate scrie un program compatibil: HTTP, SMTP, DNS. Altele sunt **proprietare** (*proprietary*), ca multe protocoale de mesagerie și de apeluri video. Observați deosebirea dintre o *aplicație* și *protocolul ei de nivel aplicație*: Web-ul este o aplicație alcătuită din documente HTML, browsere, servere web și standarde; HTTP este doar protocolul prin care browserele și serverele schimbă mesaje.

6. Web-ul: obiecte și URL-uri

O pagină web este alcătuită din **obiecte** (*objects*): fișiere precum un document HTML, imagini, foi de stil, scripturi și videoclipuri, fiecare adresabil printr-un **URL** (*Uniform Resource Locator*). Cele mai multe pagini sunt un **fișier HTML de bază** (*base HTML file*) care face referire la multe alte obiecte; o pagină obișnuită de știri face astăzi referire la mult peste o sută.

```
https://www.example.org:8443/courses/networks/week3.html?lang=en#exercises
\___/   \_____________/ \__/\_____________________________/ \_____/ \_______/
schemă      gazdă        port            cale              interogare fragment
```

Schema (*scheme*) numește protocolul (http, https). **Gazda** (*host*) este tradusă de DNS într-o adresă IP. **Portul** este opțional și are implicit valoarea 80 pentru http și 443 pentru https. **Calea** (*path*) numește obiectul de pe server, **interogarea** (*query*) transmite parametri, iar **fragmentul** (*fragment*) este folosit doar de browser (pentru a derula la o parte a paginii) — nu este trimis niciodată serverului.

HTTP (*HyperText Transfer Protocol*) este protocolul de nivel aplicație al Web-ului. Clientul (browserul) trimite o **cerere** (*request*) pentru un obiect; serverul întoarce un **răspuns** (*response*) care îl conține. HTTP/1.1 și HTTP/2 rulează peste TCP; HTTP/3 rulează peste QUIC. HTTP este **fără stare** (*stateless*): serverul nu păstrează nicio amintire despre cererile anterioare ale aceluiași client. Lipsa stării face serverele mai simple și mai robuste; când o aplicație are nevoie de stare — o autentificare, un coș de cumpărături — aceasta se adaugă deasupra, cu **cookie-uri** (*cookies*) (săptămâna viitoare).

7. Conexiunile HTTP și costul lor în timpi dus-întors

Câte conexiuni TCP ar trebui să folosească un browser ca să aducă o pagină și obiectele ei? Există două răspunsuri de bază, iar diferența dintre ele se măsoară în **timpi dus-întors** (*round-trip time*, RTT) — timpul în care un pachet mic ajunge de la client la server și înapoi.

7.1 Conexiuni nepersistente

O conexiune TCP pentru fiecare obiect (*non-persistent connections*): se deschide conexiunea, se trimite cererea, se primește răspunsul, se închide. Fiecare obiect costă **un RTT pentru stabilirea conexiunii TCP** (clientul trimite SYN și așteaptă SYN-ACK) plus **un RTT pentru cerere și începutul răspunsului**, plus timpul de transmisie al obiectului:

timp per obiect = 2 RTT + timpul de transmisie

Așa funcționa HTTP/1.0. Pe lângă timpii dus-întors suplimentari, fiecare conexiune costă memorie pe server, iar începutul fiecărei conexiuni este lent, cât timp TCP sondează rata disponibilă (vom vedea de ce în cursurile despre transport).

7.2 Conexiuni persistente

Serverul lasă conexiunea TCP deschisă după ce răspunde, iar cererile ulterioare dintre același client și același server o refolosesc (*persistent connections*). După primul obiect, fiecare obiect în plus costă **un RTT** plus timpul lui de transmisie. Conexiunile persistente sunt implicite în HTTP/1.1. Conexiunea se închide când oricare parte trimite `Connection: close` sau după o perioadă de inactivitate.

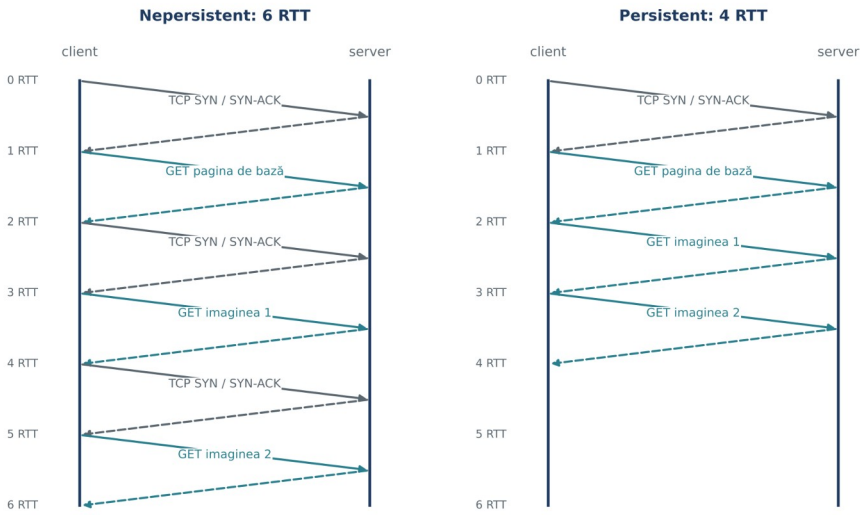


Figura 1. Aducerea unei pagini de bază și a două imagini. Nepersistent: 2 RTT per obiect. Persistent: o singură stabilire a conexiunii, apoi 1 RTT per obiect. Timpii de transmisie sunt ignorați.

7.3 Mai multe lucruri deodată

- **Conexiuni paralele** (*parallel connections*). Browserele deschid simultan mai multe conexiuni TCP către același server (de obicei până la șase pentru HTTP/1.1) și aduc mai multe obiecte deodată. Asta ascunde latența, dar multiplică încărcarea serverului și concurează inechitabil cu conexiunile unice ale altor utilizatori.
- **Cereri în bandă** (*pipelining*, în romgleză „pipelining”). HTTP/1.1 permite unui client să trimită mai multe cereri pe o conexiune fără să aștepte răspunsurile. Răspunsurile trebuie să se

întoarcă în ordine, așa că un răspuns lent le blochează pe cele din spatele lui — **blocarea la capul cozii** (*head-of-line blocking*). În practică, browserele au dezactivat pipelining-ul.

- **Multiplexare** (*multiplexing*). HTTP/2 împarte cererile și răspunsurile în cadre (*frames*) și le întrețese pe o singură conexiune TCP, eliminând blocarea la capul cozii la nivelul HTTP. HTTP/3 o elimină și la nivelul transport, rulând peste QUIC. Amândouă vin săptămâna viitoare.

Exemplu rezolvat

O pagină are un fișier HTML de bază și 10 imagini mici pe același server; RTT = 50 ms; se ignoră transmisia și DNS.

Nepersistent, câte una: 11 obiecte × 2 RTT = 22 RTT = **1100 ms**.

Nepersistent, 5 conexiuni paralele pentru imagini: baza 2 RTT, apoi două runde de imagini × 2 RTT = 6 RTT = **300 ms**.

Persistent, câte o cerere: 2 RTT pentru pagina de bază (inclusiv stabilirea conexiunii) + 10 × 1 RTT = 12 RTT = **600 ms**.

Persistent cu pipelining (sau HTTP/2): 2 RTT pentru pagina de bază + 1 RTT pentru toate imaginile = 3 RTT = **150 ms**.

La paginile alcătuite din multe obiecte mici, timpul dus-întors, nu rata legăturii, domină încărcarea.

8. Mesajele HTTP/1.1

Mesajele HTTP/1.1 sunt **text**: linii de caractere ASCII, fiecare terminată cu retur de car și avans de linie (*carriage return, line feed*, CR LF, scris `\r\n`). De aceea ați putut tasta o cerere de mână în Laboratorul 1. Specificațiile actuale sunt RFC 9110 (semantica HTTP, comună tuturor versiunilor) și RFC 9112 (formatul mesajelor HTTP/1.1), ambele din 2022.

8.1 Cereri

```
GET /courses/networks/week3.html HTTP/1.1     <- linia de cerere: metodă, țintă, versiune
Host: www.example.org                          <- linii de antet: nume: valoare
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
Accept: text/html
Accept-Language: ro, en;q=0.8
Connection: keep-alive
                                               <- linie goală: sfârșitul antetului
```

Linia de cerere (*request line*) dă metoda, ținta (calea și interogarea din URL) și versiunea. Urmează **liniile de antet** (*header lines*), câte una pe rând. **Antetul Host** este obligatoriu în HTTP/1.1: un singur server, cu o singură adresă IP, poate găzdui mii de site-uri, iar antetul Host îi spune la care se referă cererea. O **linie goală** încheie antetul. Unele cereri poartă apoi un **corp** (*body*) — de exemplu datele unui formular trimis.

Metodă	Semnificație	Sigură	Idempotentă
GET	Aduce obiectul numit de țintă	da	da
HEAD	Ca GET, dar întoarce doar antetul (fără corp)	da	da
POST	Trimite date spre prelucrare (un formular, o comandă, o încărcare)	nu	nu
PUT	Creează sau înlocuiește obiectul de la țintă cu corpul cererii	nu	da
DELETE	Șterge obiectul de la țintă	nu	da

Metodă	Semnificație	Sigură	Idempotentă
PATCH	Modifică o parte a unui obiect	nu	nu (în general)
OPTIONS	Întreabă ce metode acceptă serverul pentru țintă	da	da

O metodă este **sigură** (*safe*) dacă nu schimbă nimic pe server și **idempotentă** (*idempotent*) dacă repetarea ei are același efect ca o singură execuție. Aceste proprietăți contează pentru întrebarea ridicată în Cursul 1: un client care nu primește răspuns poate repeta fără risc o cerere idempotentă, dar repetarea unui POST poate plasa o comandă de două ori. De aceea browserele întreabă „Confirmați retrimiterea formularului?” când reîncărcați o pagină obținută prin POST.

8.2 Răspunsuri

```
HTTP/1.1 200 OK                                  <- linia de stare: versiune, cod, motiv
Date: Mon, 19 Oct 2026 08:00:00 GMT
Server: nginx
Last-Modified: Fri, 16 Oct 2026 14:12:30 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 5120

<!doctype html> ... (5120 de octeți de corp)
```

Răspunsul începe cu **linia de stare** (*status line*), care conține un **cod de stare** (*status code*) din trei cifre; prima cifră dă clasa răspunsului:

Clasă	Semnificație	Coduri frecvente
1xx	informare	100 Continue · 101 Switching Protocols
2xx	succes	**200 OK** · 201 Created · 204 No Content
3xx	redirecționare: căutați în altă parte	**301 Moved Permanently** · 302 Found · **304 Not Modified**
4xx	eroare a clientului	400 Bad Request · 401 Unauthorized · 403 Forbidden · **404 Not Found** · 405 Method Not Allowed
5xx	eroare a serverului	500 Internal Server Error · 502 Bad Gateway · 503 Service Unavailable

8.3 Unde se termină un mesaj?

TCP livrează un **flux de octeți** (*byte stream*) fără granițe între mesaje (secțiunea 9). Prin urmare, receptorul are nevoie de o regulă ca să știe unde se termină fiecare mesaj HTTP — altfel, pe o conexiune persistentă, nu poate deosebi sfârșitul unui răspuns de începutul următorului. HTTP/1.1 folosește trei reguli:

1. **Antetul se termină la prima linie goală.** Receptorul citește linii până găsește `\r\n\r\n`.
2. **Un corp de lungime cunoscută** este anunțat prin **Content-Length**: receptorul citește exact atâția octeți după antet.
3. **Un corp de lungime necunoscută** — generat chiar în timp ce este trimis — folosește **Transfer-Encoding: chunked** (codare în bucăți, în romgleză „chunked”): corpul este trimis în bucăți (*chunks*), fiecare precedată de lungimea ei în hexazecimal, pe o linie separată, și încheiat cu o bucată de lungime 0. (Un server poate și să închidă pur și simplu conexiunea la sfârșitul corpului, dar atunci conexiunea nu mai poate fi refolosită.)

```
HTTP/1.1 200 OK
Content-Type: text/plain
Transfer-Encoding: chunked

7
Network
9
s, week 3
0
```

Corpul de mai sus este alcătuit din cei 16 octeți „Networks, week 3”. Orice protocol construit peste TCP trebuie să rezolve aceeași problemă de **încadrare** (*framing*). O veți întâlni astăzi la laborator și o veți rezolva în trei feluri: mesaje de lungime fixă, linii terminate cu `\n` și un prefix de lungime.

9. Interfața socket, pas cu pas

Interfața **Berkeley sockets** a apărut în 4.2BSD UNIX în 1983 și este astăzi interfața oricărui sistem de operare; Windows o implementează ca **Winsock**, cu mici diferențe pe care le enumeră fișa laboratorului Windows. Figura 2 arată apelurile făcute de un server TCP și de un client TCP.

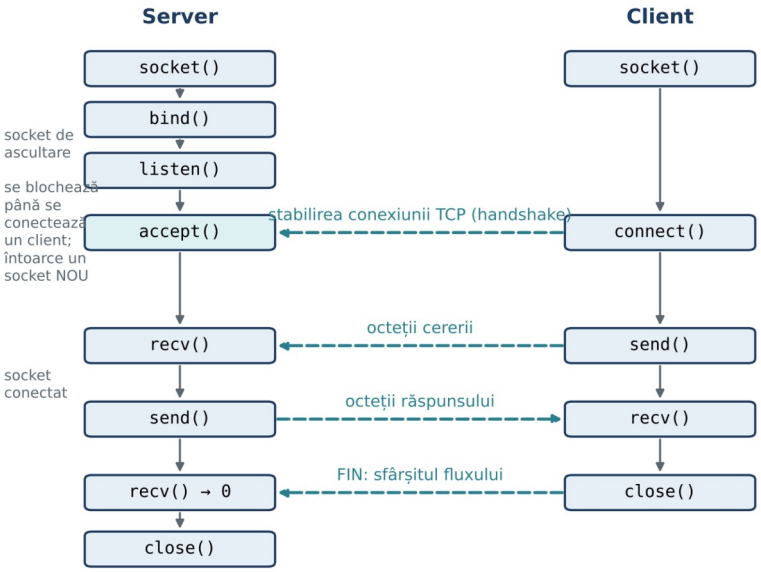


Figura 2. Apelurile de sistem ale unui server și ale unui client TCP. accept() întoarce un socket nou, conectat, pentru fiecare client; socket-ul de ascultare continuă să aștepte alți clienți.

Apel	Cine	Ce face
`socket()`	ambii	Creează un socket: familia de adrese (IPv4/IPv6) și tipul (flux = TCP, datagramă = UDP). Întoarce un descriptor.
`bind()`	serverul	Leagă socket-ul de o adresă și un port local, de exemplu 0.0.0.0:8080.
`listen()`	serverul	Marchează socket-ul ca pasiv: va accepta conexiuni. Stabilește lungimea cozii de conexiuni în așteptare.
`connect()`	clientul	Pornește stabilirea conexiunii TCP cu adresa și portul serverului. Sistemul de operare alege un port local efemer.
`accept()`	serverul	Așteaptă o conexiune încheiată și întoarce pentru ea un **socket nou**; socket-ul de ascultare rămâne deschis.

Apel	Cine	Ce face
`send()` / `recv()`	ambii	Scriu octeți în conexiune și citesc octeți din ea. `recv()` întoarce 0 când cealaltă parte a închis.
`close()`	ambii	Eliberează socket-ul; pentru TCP, trimite FIN ca să încheie fluxul în acest sens.

9.1 Trei fapte care îi surprind pe începători

- **Un flux nu are granițe între mesaje.** Două apeluri `send()` de câte 5 octeți pot sosi ca un singur `recv()` de 10 octeți sau ca apeluri `recv()` de 3 și 7. Un singur `send()` de 300 000 de octeți poate sosi în cinci bucăți de dimensiuni diferite. Un program trebuie să ruleze în buclă până are octeții pe care îi așteaptă, iar protocolul trebuie să definească unde se termină un mesaj.
- **`send()` poate accepta mai puțini octeți decât i s-au cerut.** Întoarce numărul de octeți copiați în memoria tampon a sistemului de operare; restul trebuie trimiși din nou. Un program corect îl învelește într-o buclă (`send_all` la laborator; `sendall` în Python).
- **Ordinea octeților contează.** Numerele de port, adresele și orice întregi binari trimiși prin rețea folosesc **ordinea de rețea a octeților** (*network byte order*, big-endian). Funcțiile `htons`, `htonl`, `ntohs`, `ntohl` convertesc între ordinea gazdei și cea a rețelei; uitarea lor este eroarea clasică din cauza căreia, pe o mașină x86, un server ascultă pe portul 36895 în loc de 8080.

Rezumat

- Aplicațiile de rețea rulează doar pe sistemele terminale și sunt organizate **client–server**, **de la egal la egal** (*peer-to-peer*) sau hibrid.
- Procesele comunică prin **socket-uri**; un proces este adresat prin **adresă IP + port**; o conexiune TCP este identificată de **cvadruplul** ei (*4-tuple*).
- Aplicațiile diferă prin nevoile lor de **fiabilitate, debit, temporizare și securitate**. **TCP** oferă un flux de octeți fiabil și ordonat, cu controlul fluxului și al congestiei; **UDP** oferă datagrame simple; **TLS** adaugă securitate; niciunul nu oferă garanții de timp.
- Un protocol de nivel aplicație definește **tipurile de mesaje, sintaxa, semantica și regulile**.
- Web-ul este alcătuit din **obiecte** numite prin **URL-uri** și aduse cu **HTTP**, un protocol cerere–răspuns **fără stare** (*stateless*).
- HTTP **nepersistent** costă 2 RTT per obiect; HTTP **persistent** costă 1 RTT per obiect după primul. Conexiunile paralele, pipelining-ul și multiplexarea reduc și mai mult timpul de încărcare a paginilor.
- Mesajele HTTP/1.1 sunt text: linie de cerere sau de stare, antete, linie goală, corp opțional. Sfârșitul corpului este dat de **Content-Length** sau de codarea **chunked** — problema de **încadrare** (*framing*) pe care trebuie să o rezolve orice protocol peste TCP.
- Serverele TCP apelează `socket`, `bind`, `listen`, `accept`; clienții `socket`, `connect`; amândoi `send`, `recv`, `close`.

Termeni-cheie

client–server · de la egal la egal (peer-to-peer) · auto-scalare (self-scaling) · proces · socket · adresă IP · număr de port · port efemer (ephemeral port) · cvadruplu (4-tuple) · transfer fiabil de date · aplicație elastică · debit (throughput) · temporizare (timing) · TCP · UDP · TLS · QUIC · protocol de nivel aplicație · protocol deschis / proprietar · obiect · URL · schemă · gazdă · cale · interogare (query)

· fragment · HTTP · fără stare (stateless) · conexiune nepersistentă / persistentă · RTT · conexiuni paralele · pipelining · blocare la capul cozii (head-of-line blocking) · multiplexare · linie de cerere · linie de antet · antetul Host · metodă · sigură (safe) · idempotentă · linie de stare · cod de stare · Content-Length · codare chunked · încadrare (framing) · Berkeley sockets · Winsock · socket de ascultare (listening socket) · socket conectat (connected socket) · ordinea de rețea a octeților (network byte order)

Întrebări de recapitulare

1. De ce poate fi lansată o aplicație de rețea nouă fără nicio schimbare în ruterele Internetului?
2. Într-o descărcare BitTorrent, care entitate este clientul și care serverul? Se poate schimba răspunsul în timpul descărcării?
3. Un server web are 10 000 de conexiuni TCP deschise, toate către portul lui 443. Cum livrează un segment sosit conexiunii potrivite?
4. Dați o aplicație pentru care UDP este o alegere mai bună decât TCP și explicați ce serviciu TCP i-ar sta în cale.
5. Nici TCP, nici UDP nu criptează. Cum protejează HTTPS o conexiune și la ce nivel?
6. Ce parte a unui URL nu este trimisă niciodată serverului? De ce?
7. De ce este obligatoriu antetul Host în HTTP/1.1?
8. Explicați de ce un browser poate repeta fără risc o cerere GET, dar ar trebui să avertizeze înainte de a repeta un POST.
9. Dați două moduri în care un receptor HTTP/1.1 poate afla unde se termină corpul unui răspuns.
10. Ce întoarce `accept()` și de ce nu este acesta chiar socket-ul de ascultare?

Exerciții

1. O pagină constă dintr-un fișier HTML de bază și 4 imagini, toate pe un singur server. RTT = 80 ms; fiecare obiect are nevoie de 10 ms pentru transmisie; se ignoră DNS. Calculați timpul de încărcare a paginii cu (a) conexiuni nepersistente, câte una; (b) conexiuni persistente fără pipelining; (c) conexiuni persistente cu pipelining.
2. Refaceți exercițiul 1(a) când browserul deschide până la 4 conexiuni nepersistente paralele pentru imagini.
3. Decodificați acest corp chunked și dați-i lungimea: `5\r\nHello\r\n8\r\n, world!\r\n0\r\n\r\n`.
4. Un client trimite o cerere de 1 000 de octeți cu un singur apel `send()`. Are serverul garanția că o primește cu un singur `recv()`? Dar dacă cererea are 10 octeți? Explicați.
5. Pe o mașină little-endian, un programator scrie `addr.sin_port = 8080;` în loc de `htons(8080)`. 8080 este 0x1F90. Pe ce port ascultă de fapt serverul?

Răspunsuri

1. (a) 5 obiecte × (2 × 80 + 10) = **850 ms**. (b) Pagina de bază: 2 × 80 + 10 = 170 ms; 4 imagini × (80 + 10) = 360 ms; total **530 ms**. (c) Pagina de bază 170 ms; cele 4 cereri pleacă împreună, primul răspuns sosește după 80 ms, iar cele patru sunt transmise una după alta: 80 + 4 × 10 = 120 ms; total **290 ms**.

2. Pagina de bază 170 ms, apoi toate cele 4 imagini în paralel, fiecare 2 × 80 + 10 = 170 ms (presupunând că transmisiile nu se încetinesc reciproc): **340 ms**.
3. „Hello” (5 octeți) urmat de „, world!” (8 octeți): corpul este „Hello, world!”, **13 octeți**.
4. Nu, în ambele cazuri. TCP este un flux de octeți; receptorul poate primi octeții în mai multe bucăți, în funcție de dimensiunile segmentelor, de temporizare și de dimensiunile memoriilor tampon. Mesajele mici sosesc *de obicei* întregi, motiv pentru care eroarea de a presupune asta trece adesea neobservată la testare și apare în producție.
5. Octeții 0x90 0x1F sunt interpretați în ordinea de rețea ca **0x901F = 36 895**.

Lecturi și pașii următori

Obligatoriu

- Kurose și Ross, *Computer Networking: A Top-Down Approach*, ediția a 8-a, capitolul 2: secțiunile despre principiile aplicațiilor de rețea și despre Web și HTTP; secțiunea despre programarea cu socket-uri.

Recomandat

- B. Hall, *Beej’s Guide to Network Programming* (gratuit online), capitolele 2–6.
- W. R. Stevens, B. Fenner, A. M. Rudoff, *UNIX Network Programming, Volume 1*, ediția a 3-a, capitolele 1, 4 și 5.
- RFC 9110, „HTTP Semantics”, secțiunile 9 (metode) și 15 (coduri de stare); RFC 9112, „HTTP/1.1”, secțiunile 2–3 și 6–7 (formatul mesajelor și încadrarea).

Laboratorul 3 (săptămâna aceasta)

Socket-uri I: construiți, rulați și „stricați” un client și un server TCP de ecou (*echo*) în C, observați cum este tăiat în bucăți un flux de octeți și implementați două metode de încadrare.

Cursul următor

Web-ul, în continuare: cookie-uri și stare, memorarea în cache (*web caching*) și GET condiționat (*conditional GET*), HTTP/2 și HTTP/3, rețelele de distribuire a conținutului (*content distribution networks*, CDN).

PARTEA B · SĂPTĂMÂNA 3

Seminarul 3: timpi HTTP, mesaje și fluxuri de octeți

Ședință de probleme de două ore (2 × 50 de minute), însoțind Cursul 3

Seminarul transformă Cursul 3 în calcule și în citirea atentă a mesajelor. Lucrați în perechi; câte o pereche prezintă fiecare problemă.

La sfârșitul seminarului ar trebui să puteți

- să alegeți un serviciu de transport pornind de la cerințele unei aplicații;
- să calculați timpii de încărcare a unei pagini pentru diferite strategii de conexiune HTTP;
- să citiți cereri și răspunsuri HTTP/1.1 și să decodificați corpuri chunked;
- să asociați situații cu coduri de stare HTTP și să judecați dacă o cerere poate fi repetată;
- să enumerați socket-urile și cvadruplele de pe un server aglomerat;
- să proiectați o regulă de încadrare pentru un protocol construit peste un flux de octeți.

Plan

Pas	Activitate	Timp
A	Problema 1: ce transport?	10 min
B	Problemele 2–3: cât durează o pagină?	25 min
C	Problema 4: citirea unui schimb HTTP	15 min
	Pauză	10 min
D	Problemele 5–6: încadrare și corpuri chunked	15 min
E	Problemele 7–8: coduri de stare și cereri repetabile	10 min
F	Problemele 9–10: socket-uri, cvadruple și proiectarea unei încadrări	15 min
	Temă	(în F)

A. Ce transport?

Problema 1

Pentru fiecare aplicație, precizați-i nevoile (toleranța la pierderi, debitul — *throughput*, temporizarea — *timing*) și alegeți TCP, UDP sau „UDP cu ceva construit deasupra”. Justificați într-o propoziție.

	Aplicație
a	Autentificare la distanță pe un server (SSH)
b	Un joc de curse cu mai mulți jucători care trimite poziția fiecărei mașini de 30 de ori pe secundă
c	Încărcarea unei copii de siguranță de 2 GB într-un serviciu cloud
d	Un apel video între două telefoane
e	Întrebarea adresată unui server DNS pentru adresa unui nume

	Aplicație
f	Un senzor care raportează temperatura în fiecare minut printr-o legătură radio cu pierderi

Rezolvare

(a) Fără pierderi, debit mic, interactiv, dar tolerant la zeci de ms: **TCP**. (b) Tolerează pierderi — o poziție veche nu mai folosește la nimic odată ce există una mai nouă — și este sensibil la timp: **UDP**; TCP ar întârzia pozițiile noi în spatele celor vechi retransmise. (c) Fără pierderi, elastic: **TCP**. (d) Tolerează pierderi, sensibil la timp, are nevoie de Mbit/s: **UDP** cu un protocol de timp real deasupra (RTP — *Real-time Transport Protocol*, ca în WebRTC). (e) O cerere mică și un răspuns; clientul reîncearcă singur dacă nu vine răspuns: **UDP** (TCP pentru răspunsuri mari). (f) O singură citire pe minut, pierderea uneia este acceptabilă: **UDP**; dacă fiecare citire contează, se adaugă deasupra confirmări (*acknowledgements*) și reîncercări sau se folosește TCP.

B. Cât durează o pagină?

Problema 2

O pagină are un fișier HTML de bază de 50 KB și 6 imagini de câte 100 KB, toate pe un singur server. Legătura de acces a clientului are 20 Mbit/s și este gâtuirea (*bottleneck*); RTT = 40 ms. Se ignoră DNS și dimensiunile antetelor; 1 KB = 1000 de octeți.

1. Care este timpul de transmisie al fișierului de bază? Al unei imagini?
2. Calculați timpul de încărcare a paginii cu conexiuni nepersistente, câte una.
3. Cu conexiuni persistente, fără pipelining.
4. Cu conexiuni persistente și pipelining (toate cererile de imagini trimise deodată).
5. Ce contează mai mult aici: RTT-ul sau rata legăturii? Cum s-ar schimba răspunsul pentru o pagină cu 60 de imagini de câte 2 KB?

Rezolvare

1. Baza: 400 000 de biți / 2×10^7 bit/s = 20 ms. O imagine: 800 000 / 2×10^7 = 40 ms.

2. Baza: 2 × 40 + 20 = 100 ms. Fiecare imagine: 2 × 40 + 40 = 120 ms; 6 imagini 720 ms. Total **820 ms**.

3. Baza 100 ms; fiecare imagine 40 + 40 = 80 ms; total 100 + 480 = **580 ms**.

4. Baza 100 ms; apoi 40 ms până ajung cererile și se întorc primii biți, plus 6 × 40 ms transmisie: 280 ms. Total **380 ms**.

5. Aici transmisia (240 ms pentru imagini) și timpii dus-întors (câte 40 ms) sunt comparabile. Cu 60 de imagini mici, fiecare are nevoie de 0,8 ms pentru transmisie: varianta nepersistentă costă 60 × (80 + 0,8) ≈ 4,8 s, cea persistentă fără pipelining 60 × 40,8 ≈ 2,4 s, iar cea cu pipelining doar aproximativ 40 + 48 = 88 ms după pagina de bază. La multe obiecte mici domină **timpii dus-întors**.

Problema 3

Un browser încarcă pagina din Problema 2 cu până la **6 conexiuni nepersistente paralele** pentru imagini. Cele șase transferuri împart în mod egal legătura de acces de 20 Mbit/s.

1. Cât durează pagina?

2. De ce răspunsul nu este de 6 ori mai bun decât în Problema 2.2?

> **Rezolvare**
>
> 1. Baza 100 ms. Apoi șase conexiuni deodată: 1 RTT pentru stabilirea conexiunii (40 ms) + 1 RTT pentru cerere și primii biți (40 ms) + transmisia a 6 × 100 KB = 4,8 Mbit pe 20 Mbit/s = 240 ms. Total 100 + 80 + 240 = **420 ms**.
> 2. Paralelismul ascunde timpii dus-întors, nu transmisia: cele șase imagini împart în continuare o singură legătură de 20 Mbit/s, așa că cei 240 ms de transmisie nu se micșorează.

C. Citirea unui schimb HTTP

Problema 4

O captură arată această cerere și acest răspuns pe o singură conexiune TCP.

```
GET /files/notes.pdf HTTP/1.1
Host: cn.example.edu
User-Agent: curl/8.5.0
If-Modified-Since: Fri, 16 Oct 2026 14:12:30 GMT
Accept: */*

HTTP/1.1 304 Not Modified
Date: Mon, 19 Oct 2026 08:00:02 GMT
Last-Modified: Fri, 16 Oct 2026 14:12:30 GMT
ETag: "5f3a-1e2b4"
```

1. Care este URL-ul complet al obiectului cerut?
2. Ce program a trimis cererea? Este conexiunea persistentă? De unde știți?
3. Ce cere antetul If-Modified-Since și ce răspunde serverul?
4. Câți octeți de corp are răspunsul? De unde știe clientul?
5. De ce ar trimite un client o asemenea cerere?

> **Rezolvare**
>
> 1. http://cn.example.edu/files/notes.pdf — schema este http, pentru că avem HTTP simplu pe o conexiune TCP (un schimb HTTPS ar fi criptat).
> 2. curl, versiunea 8.5.0 (antetul User-Agent). Persistentă: HTTP/1.1 este persistent în mod implicit și nicio parte nu a trimis `Connection: close`.
> 3. „Trimite obiectul doar dacă s-a schimbat după 16 octombrie, 14:12:30.” Serverul răspunde **304 Not Modified**: copia clientului este încă valabilă.
> 4. Niciunul: un răspuns 304 nu are niciodată corp; antetul se termină la linia goală.
> 5. Clientul are o copie în cache (*cached copy*) și vrea să verifice dacă este încă proaspătă fără să o descarce din nou. Acest **GET condiționat** (*conditional GET*) este subiectul cursului de săptămâna viitoare despre memorarea în cache.

D. Încadrare

Problema 5 — corpuri chunked

Decodificați corpul următor (`\r\n` marchează CR LF — *carriage return, line feed*). Ce lungime are și ce înseamnă ultima linie?

```
1a\r\n
abcdefghijklmnopqrstuvwxyz\r\n
a\r\n
0123456789\r\n
0\r\n
\r\n
```

> **Rezolvare**
> Dimensiunile bucăților (*chunks*) sunt în hexazecimal: 0x1a = 26 de octeți (alfabetul), apoi 0xa = 10 octeți (cifrele). Total **36 de octeți**. Bucata de dimensiune 0, urmată de o linie goală, încheie corpul; conexiunea poate transporta acum răspunsul următor.

Problema 6 — ce vede receptorul

Un client apelează `send(fd, "HELLO", 5)` și imediat după aceea `send(fd, "WORLD", 5)` pe un socket TCP. Serverul apelează repetat `recv(fd, buf, 100)`.

1. Enumerați trei șiruri diferite de valori întoarse pe care le-ar putea observa serverul.
2. Proiectantul protocolului voia două mesaje separate. Propuneți două moduri de a face granița vizibilă pentru server și dați octeții exacți trimiși în fiecare caz.

> **Rezolvare**
> 1. De exemplu: 10 („HELLOWORLD” într-un singur apel); 5 și 5; 3 și 7 („HEL”, „LOWORLD”). Este posibilă orice împărțire a celor 10 octeți, în ordine — dar niciodată o reordonare sau o pierdere. 2. **Delimitator** (*delimiter*): fiecare mesaj se încheie cu un caracter de linie nouă: `HELLO\nWORLD\n` (12 octeți); serverul citește până la fiecare `\n`. Delimitatorul nu trebuie să apară niciodată în interiorul unui mesaj sau trebuie „scăpat” (*escaped*). **Prefix de lungime** (*length prefix*): înaintea fiecărui mesaj se trimite lungimea lui pe 4 octeți, în ordinea de rețea: `00 00 00 05 H E L L O 00 00 00 05 W O R L D` (18 octeți); serverul citește 4 octeți, apoi exact atâția câți indică aceștia. Funcționează pentru orice conținut, inclusiv date binare.

E. Coduri de stare și cereri repetabile

Problema 7

Ce cod de stare (*status code*) ar trebui să trimită serverul în fiecare situație?

1. Site-ul s-a mutat definitiv de pe http:// pe https://.
2. Un client cere /admin fără să se fi autentificat.
3. Un student autentificat cere /admin, pe care doar profesorii au voie să o vadă.
4. Clientul trimite `DELETE /index.html` unui server care nu șterge niciodată pagini.
5. Un cont nou a fost creat printr-o cerere POST.
6. Aplicația din spatele serverului web s-a prăbușit în timp ce trata cererea.
7. Linia de cerere a clientului este `GTE / HTTP/1.1`.

> **Rezolvare**
> 1. **301 Moved Permanently** (cu un antet Location). 2. **401 Unauthorized** (este necesară autentificarea). 3. **403 Forbidden** (autentificat, dar fără drept de acces). 4. **405 Method Not Allowed** (sau 501 Not Implemented, dacă serverul nu cunoaște deloc metoda). 5. **201 Created**. 6. **500 Internal Server Error** (sau 502 Bad Gateway, dacă eroarea este raportată de un proxy aflat în față). 7. **400 Bad Request** — sau 501, pentru că GTE este o metodă necunoscută.

Problema 8

O aplicație mobilă trimite o cerere, iar conexiunea cade înainte să sosească vreun răspuns. Pentru fiecare cerere: poate aplicația să o repete automat?

1. `GET /timetable?group=CS2`
2. `PUT /profile/photo` cu o fotografie nouă
3. `DELETE /cart/items/17`
4. `POST /orders` cu conținutul coșului de cumpărături

> **Rezolvare**
> 1. Da: sigură și idempotentă. 2. Da: idempotentă — încărcarea aceleiași fotografii de două ori lasă același rezultat. 3. Da: idempotentă — al doilea DELETE nu mai găsește nimic de șters (poate răspunde 404, pe care aplicația trebuie să îl accepte). 4. **Nu**: prima comandă poate să fi fost plasată; repetarea ar putea comanda de două ori. Soluția obișnuită este o **cheie de idempotență** (*idempotency key*): aplicația trimite odată cu comanda un identificator unic, iar serverul ignoră o a doua comandă cu același identificator — identificatorul de cerere din Seminarul 1, Problema 6.

F. Socket-uri

Problema 9

Un server web de la adresa 203.0.113.10 ascultă pe portul 80. Sunt deschise trei conexiuni: două de la un laptop cu adresa 198.51.100.7 (două file — *tabs* — de browser) și una de la un telefon cu adresa 192.0.2.33.

1. Câte socket-uri are deschise procesul server?
2. Scrieți cvadruple (*4-tuples*) plauzibile pentru cele trei conexiuni, din punctul de vedere al serverului.
3. Cele două conexiuni de la laptop au aceeași adresă IP sursă și aceeași adresă IP și același port de destinație. Cum le deosebește serverul?

> **Rezolvare**
> 1. Patru: socket-ul de ascultare și câte un socket conectat pentru fiecare client, așa cum le întoarce `accept()`.
> 2. De exemplu (IP sursă, port sursă, IP destinație, port destinație): (198.51.100.7, 52314, 203.0.113.10, 80); (198.51.100.7, 52316, 203.0.113.10, 80); (192.0.2.33, 40122, 203.0.113.10, 80).
> 3. După portul sursă: sistemul de operare al laptopului a dat fiecărui socket un alt port efemer. Două conexiuni nu pot avea niciodată toate cele patru numere identice.

Problema 10 — proiectare

Trebuie să proiectați protocolul unui mic serviciu de încărcare de fișiere peste TCP. Clientul trimite un nume de fișier și octeții fișierului; serverul răspunde OK sau o eroare. Fișierul poate conține orice octeți, inclusiv caractere de linie nouă și zerouri. Specificați formatul exact al mesajelor, astfel încât doi studenți să poată implementa independent clientul și serverul.

Un răspuns posibil

Cerere: o linie de antet, ca text, apoi octeții bruți: PUT `<name> <size>\r\n` urmată de exact `<size>` octeți. Numele nu poate conține spații sau CR LF; dimensiunea este în zecimal.

Răspuns: o linie: OK`\r\n` sau ERR `<code> <text>\r\n` (de exemplu ERR `413 too large`).

Încadrare: antetul se termină la primul CR LF (un delimitator, sigur pentru că numele nu îl pot conține); sfârșitul corpului este dat de lungimea lui (un câmp de lungime, sigur pentru date binare) — exact cele două reguli ale HTTP/1.1.

Reguli: pe o conexiune pot urma mai multe cereri una după alta; serverul răspunde fiecăreia în ordine; oricare parte poate închide după un răspuns. Repetarea unui PUT cu același fișier nu face niciun rău: cererea este idempotentă.

Temă

- Rezolvați exercițiile 1–5 de la sfârșitul Cursului 3 fără să vă uitați la răspunsuri.
- Cu `curl -v http://example.org/` și apoi cu `curl -v --http1.0 http://example.org/`, comparați cele două cereri și comportamentul conexiunii (antetul Connection).
- Scrieți, pentru protocolul din Problema 10, ce ar trebui să facă serverul dacă conexiunea se închide înainte să fi sosit `<size>` octeți.

PARTEA C · SĂPTĂMÂNA 3

Laboratorul 3 (Linux): Socket-uri I — un client și un server TCP în C

Fișă de laborator de două ore (2 × 50 de minute), însoțind Cursul 3

Astăzi scrieți programele pe care până acum doar le-ați observat. Veți construi un client și un server TCP în C, veți vedea la lucru apelurile din Cursul 3, veți vedea cum un flux de octeți (*byte stream*) se rupe în bucăți și veți implementa două feluri de a reface granițele dintre mesaje. Toate programele din această fișă se compilează fără avertismente și au fost testate pe Ubuntu 24.04; ele sunt furnizate și ca fișiere. Codul sursă și comentariile din el sunt lăsate în engleză, ca în fișierele din arhivă.

La sfârșitul laboratorului ar trebui să puteți

- să scrieți, să compilați și să rulați un client TCP și un server TCP iterativ în C;
- să traduceți nume în adrese cu `getaddrinfo` și să tratați IPv4 și IPv6 într-un client;
- să observați cu `ss` socket-urile de ascultare și pe cele conectate, împreună cu cvadruplele lor;
- să explicați, cu măsurători, de ce `recv` trebuie apelat în buclă;
- să implementați încadrarea pe linii și cea cu prefix de lungime;
- să recunoașteți erorile frecvente: conexiune refuzată, adresă deja folosită, cealaltă parte a închis, conductă întreruptă (*broken pipe*).

Înainte de a începe

- **Mediul de lucru.** Mașina virtuală a cursului sau orice Linux cu un compilator C. Instalați o singură dată uneltele:

```
sudo apt install -y build-essential netcat-openbsd manpages-dev manpages-posix-dev
```

- **Fișiere.** Dezarhivați arhiva acestui laborator într-un dosar `lab3/`; ea conține `netio.h`, `echo_client.c`, `echo_server.c`, `upper_server.c`, `blast.c`, `msg_client.c`, `msg_server.c` și un `Makefile`. Construiți totul cu `make`.
- **Paginile de manual.** `man 2 socket`, `man 2 bind`, `man 2 accept`, `man 2 recv`, `man 3 getaddrinfo`, `man 7 tcp`. Citiți secțiunile RETURN VALUE și ERRORS: acolo eșuează programele de rețea.
- **Porturi.** Folosiți un port între 5000 și 60000. Dacă un coleg de pe aceeași mașină îl folosește pe al vostru, alegeți altul.
- **Ce predați.** Sursele modificate și un scurt raport cu răspunsurile la **Q1–Q16**.

Plan

Pas	Activitate	Timp
1	Încălzire cu netcat: un server fără să scrieți cod	10 min
2	Citiți și rulați clientul de ecou	15 min
3	Citiți și rulați serverul de ecou	20 min
4	Socket-urile văzute de sistemul de operare	5 min

Pas	Activitate	Timp
	Pauză	10 min
5	Un flux nu este un șir de mesaje	15 min
6	Încadrare pe linii: serverul de majuscule	10 min
7	Încadrare cu prefix de lungime	15 min
8	Stricăm lucrurile intenționat	10 min

Sarcina 1 — Un server fără să scrieți cod

În terminalul A porniți un `nc` care ascultă; din terminalul B conectați-vă la el; tastați în oricare fereastră:

```
nc -l 5000                # terminal A: listen on port 5000
nc localhost 5000         # terminal B: connect
ss -tln | grep 5000       # terminal C: listening sockets
ss -tn  | grep 5000       # terminal C: connected sockets and their 4-tuples
```

- **Q1.** Ce arată `ss -tln` înainte să se conecteze clientul și ce arată `ss -tn` după aceea? Scrieți cvadruplul (*4-tuple*) conexiunii. Ce port a primit clientul și cine l-a ales?
- **Q2.** Închideți terminalul B cu Ctrl+D. Ce se întâmplă cu serverul? Ce mesaj TCP l-a anunțat?

Sarcina 2 — Clientul de ecou

Toate programele folosesc funcțiile ajutătoare din `netio.h`. `send_all` și `recv_all` rulează în buclă până când toți octeții ceruți au fost scriși sau citiți; `send_msg` și `recv_msg` sunt folosite în Sarcina 7.

```
/* netio.h - helpers for TCP byte streams (Computer Networks, Lab 3) */
#ifndef NETIO_H
#define NETIO_H
#include <stdint.h>
#include <errno.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

/* Write exactly len bytes. Returns 0 on success, -1 on error. */
static inline int send_all(int fd, const void *buf, size_t len) {
    const char *p = buf;
    while (len > 0) {
        ssize_t n = send(fd, p, len, 0);
        if (n < 0) { if (errno == EINTR) continue; return -1; }
        p += n; len -= (size_t)n;
    }
    return 0;
}

/* Read exactly len bytes. Returns 1 on success, 0 if the peer closed first, -1 on error.
*/
static inline int recv_all(int fd, void *buf, size_t len) {
    char *p = buf;
    while (len > 0) {
        ssize_t n = recv(fd, p, len, 0);
        if (n < 0) { if (errno == EINTR) continue; return -1; }
        if (n == 0) return 0;
        p += n; len -= (size_t)n;
```

```
    }
    return 1;
}

/* Length-prefixed messages: 4-byte length in network byte order, then the payload. */
static inline int send_msg(int fd, const void *buf, uint32_t len) {
    uint32_t netlen = htonl(len);
    if (send_all(fd, &netlen, sizeof netlen) < 0) return -1;
    return send_all(fd, buf, len);
}

/* Returns the payload length, 0 if the peer closed, -1 on error or if the message
exceeds cap. */
static inline long recv_msg(int fd, void *buf, uint32_t cap) {
    uint32_t netlen;
    int r = recv_all(fd, &netlen, sizeof netlen);
    if (r <= 0) return r;
    uint32_t len = ntohl(netlen);
    if (len > cap) return -1;
    r = recv_all(fd, buf, len);
    return r <= 0 ? -1 : (long)len;
}
#endif
```

Clientul traduce numele serverului cu `getaddrinfo`, încearcă fiecare adresă primită (IPv6 și IPv4) până când `connect` reușește, apoi trimite fiecare linie citită de la tastatură și afișează ecoul:

```
/* echo_client.c - send each line of standard input to a TCP server, print what comes
back.
   Usage: ./echo_client HOST PORT */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <netdb.h>
#include <sys/socket.h>
#include "netio.h"

int main(int argc, char *argv[]) {
    if (argc != 3) { fprintf(stderr, "usage: %s HOST PORT\n", argv[0]); return 1; }

    struct addrinfo hints = {0}, *res, *ai;
    hints.ai_family = AF_UNSPEC;          /* IPv4 or IPv6 */
    hints.ai_socktype = SOCK_STREAM;      /* TCP */
    int err = getaddrinfo(argv[1], argv[2], &hints, &res);
    if (err) { fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(err)); return 1; }

    int fd = -1;
    for (ai = res; ai != NULL; ai = ai->ai_next) {          /* try each address in turn
*/
        fd = socket(ai->ai_family, ai->ai_socktype, ai->ai_protocol);
        if (fd < 0) continue;
        if (connect(fd, ai->ai_addr, ai->ai_addrlen) == 0) break;
        close(fd); fd = -1;
    }
    freeaddrinfo(res);
    if (fd < 0) { perror("connect"); return 1; }

    char line[1024], reply[1024];
    while (fgets(line, sizeof line, stdin) != NULL) {
        size_t len = strlen(line);
        if (send_all(fd, line, len) < 0) { perror("send"); break; }
        /* The echo has the same length as what we sent, but may arrive in pieces. */
        int r = recv_all(fd, reply, len);
```

```
        if (r <= 0) { fprintf(stderr, r == 0 ? "server closed the connection\n" : "recv
failed\n"); break; }
        fwrite(reply, 1, len, stdout);
    }
    close(fd);
    return 0;
}
```

Testați-l mai întâi cu `nc -l 5000` din Sarcina 1 (tastați ecoul de mână în fereastra `nc`), apoi mergeți mai departe.

- **Q3.** De ce apelează clientul `recv_all(fd, reply, len)` și nu un singur `recv`? Ce ar merge prost cu un singur apel?
- **Q4.** Rulați `./echo_client localhost 5000` când nimic nu ascultă pe portul 5000. Ce eroare primiți și ce mesaj TCP a provocat-o?

Sarcina 3 — Serverul de ecou

```
/* echo_server.c - iterative TCP echo server: one client at a time.
   Usage: ./echo_server PORT */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include "netio.h"

int main(int argc, char *argv[]) {
    if (argc != 2) { fprintf(stderr, "usage: %s PORT\n", argv[0]); return 1; }

    setvbuf(stdout, NULL, _IOLBF, 0);                          /* print each line at once
*/
    int lfd = socket(AF_INET, SOCK_STREAM, 0);                /* 1. create */
    if (lfd < 0) { perror("socket"); return 1; }
    int on = 1;
    setsockopt(lfd, SOL_SOCKET, SO_REUSEADDR, &on, sizeof on); /* allow quick restarts */

    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);                  /* all local addresses */
    addr.sin_port = htons((uint16_t)atoi(argv[1]));            /* host -> network byte
order */
    if (bind(lfd, (struct sockaddr *)&addr, sizeof addr) < 0) { perror("bind"); return 1;
} /* 2. bind */
    if (listen(lfd, 8) < 0) { perror("listen"); return 1; }  /* 3. listen, backlog 8 */
    printf("listening on port %s\n", argv[1]);

    for (;;) {
        struct sockaddr_in peer; socklen_t plen = sizeof peer;
        int cfd = accept(lfd, (struct sockaddr *)&peer, &plen); /* 4. accept: a NEW
socket */
        if (cfd < 0) { perror("accept"); continue; }
        char ip[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &peer.sin_addr, ip, sizeof ip);
        printf("connection from %s:%u\n", ip, ntohs(peer.sin_port));

        char buf[65536]; ssize_t n; long total = 0;
        while ((n = recv(cfd, buf, sizeof buf, 0)) > 0) {      /* 5. read what has
arrived */
            printf("  recv returned %zd bytes\n", n);
            total += n;
```

```
            if (send_all(cfd, buf, (size_t)n) < 0) { perror("send"); break; }
        }
        printf("client closed after %ld bytes\n", total);
        close(cfd);                                             /* 6. close this
connection */
    }
}
```

```
./echo_server 5000                      # terminal A
./echo_client localhost 5000            # terminal B: type a few lines
./echo_client localhost 5000            # terminal C, while B is still connected
```

- **Q5.** Asociați fiecare pas numerotat al serverului cu apelurile din Figura 2 a Cursului 3. Ce afișează serverul pentru fiecare client și de unde vine numărul de port al clientului?
- **Q6.** Al doilea client (terminalul C) se conectează, dar nu primește ecou până nu iese primul. De ce? Este conexiunea lui TCP stabilită cât timp așteaptă? Verificați cu `ss -tn`. (Acesta este prețul unui server *iterativ*; în Laboratorul 5 construim servere concurente.)
- **Q7.** Scoateți `htons` din linia care setează `sin_port`, reconstruiți și rulați serverul cu portul 5000. Pe ce port ascultă de fapt (`ss -tln`)? Explicați numărul.

Sarcina 4 — Socket-urile văzute de sistemul de operare

```
ss -tnp | grep -E '5000|echo'           # -p shows the owning process
ls -l /proc/$(pgrep -n echo_server)/fd  # a socket is a file descriptor
```

- **Q8.** Cât timp este conectat un client, câte socket-uri deține `echo_server` și în ce stări? Ce numere de descriptor de fișier (*file descriptor*) au?

Sarcina 5 — Un flux nu este un șir de mesaje

`blast` trimite N octeți cu **un singur** apel `send_all`, apoi își închide sensul de trimitere. Rulați-l către serverul de ecou și urmăriți ce afișează serverul:

```
./blast localhost 5000 300000

connection from 127.0.0.1:51304
  recv returned 65482 bytes
  recv returned 65536 bytes
  recv returned 65536 bytes
  recv returned 65536 bytes
  recv returned 37910 bytes
client closed after 300000 bytes
```

O rulare reală pe interfața de buclă locală (loopback); dimensiunile voastre vor diferi de la o rulare la alta.

- **Q9.** Repetați de trei ori și cu N = 10, 5000 și 3 000 000. Un singur send — câte recv-uri? Sunt dimensiunile mereu aceleași? Ce limitează dimensiunea unui singur `recv`?
- **Q10.** Schimbați `sizeof buf` din server în 7 și trimiteți linia `hello world` din `echo_client`. Ce afișează acum serverul? Primește clientul tot ecoul complet — și de ce?

Sarcina 6 — Încadrare pe linii

`upper_server` implementează un protocol pe linii: pentru fiecare linie terminată cu `\n` răspunde cu linia scrisă cu majuscule. Bucla lui păstrează o memorie tampon (*buffer*), pentru că un `recv` poate conține o jumătate de linie, o linie întreagă sau mai multe linii:

```
    for (;;) {
        int cfd = accept(lfd, NULL, NULL);
```

```
        if (cfd < 0) continue;
        char buf[4096]; size_t used = 0; ssize_t n;
        while ((n = recv(cfd, buf + used, sizeof buf - used, 0)) > 0) {
            used += (size_t)n;
            char *nl;
            /* A single recv may hold part of a line, one line, or several lines. */
            while ((nl = memchr(buf, '\n', used)) != NULL) {
                size_t linelen = (size_t)(nl - buf) + 1;
                for (size_t i = 0; i < linelen; i++) buf[i] = (char)toupper((unsigned
char)buf[i]);
                if (send_all(cfd, buf, linelen) < 0) break;
                memmove(buf, buf + linelen, used - linelen);   /* keep the rest for later
*/
                used -= linelen;
            }
            if (used == sizeof buf) { const char *e = "ERR line too long\n";
send_all(cfd, e, strlen(e)); break; }
        }
        close(cfd);

./upper_server 5001
printf 'abc\ndef\nghi' | nc -N localhost 5001
```

- **Q11.** De ce nu primește niciodată „ghi” răspuns? Ce ar trebui să spună un protocol despre o ultimă linie incompletă?
- **Q12.** Ce se întâmplă dacă un client trimite o linie mai lungă de 4096 de octeți? Este comportamentul serverului o decizie bună de protocol? Propuneți una mai bună.

Sarcina 7 — Încadrare cu prefix de lungime

Liniile nu funcționează pentru date binare, care pot conține `\n`. Soluția generală este să trimiteți mai întâi lungimea fiecărui mesaj (*length prefix*). Studiați `send_msg` și `recv_msg` din `netio.h`, apoi rulați perechea furnizată:

```
./msg_server 5002
./msg_client localhost 5002 short "a longer message" x

short -> OK 5
a longer message -> OK 16
x -> OK 1
```

- **Q13.** Clientul trimite toate cele trei mesaje înainte să citească vreun răspuns. Cum știe serverul unde se termină fiecare, chiar dacă toți cei 34 de octeți sosesc într-un singur `recv`? De ce folosește `send_msg` funcția `htonl`?
- **Q14.** Scrieți propriul program `file_client.c`, care trimite un fișier întreg ca un singur mesaj cu `send_msg`, și verificați cu `msg_server` că dimensiunea raportată este egală cu dimensiunea fișierului (`stat -c %s file`). Încercați un fișier mai mare de 1 MB: ce se întâmplă și de ce?

Sarcina 8 — Stricăm lucrurile intenționat

Experiment	Ce faceți	Ce observați
Conexiune refuzată	Vă conectați la un port pe care nu ascultă nimic.	`connect: Connection refused` — gazda a răspuns cu un TCP RST (*reset*).
Adresă deja folosită	Opriți serverul cu Ctrl+C cât timp un client este conectat, comentați linia `setsockopt(... SO_REUSEADDR ...)`, reconstruiți și reporniți imediat.	`bind: Address already in use` timp de aproximativ un minut: vechea conexiune este în starea TIME_WAIT (`ss -tan | grep 5000`).

Experiment	Ce faceți	Ce observați
Cealaltă parte a închis	Opriți clientul cu Ctrl+C cât timp serverul așteaptă în `recv`.	`recv` întoarce 0; serverul afișează „client closed”.
Conductă întreruptă (*broken pipe*)	Scrieți un client care scrie într-o conexiune după ce serverul a închis-o.	Programul este oprit de semnalul SIGPIPE; cu `signal(SIGPIPE, SIG_IGN)`, `send` eșuează în schimb cu EPIPE.

- **Q15.** Pentru fiecare experiment, copiați textul exact al erorii și numiți mecanismul TCP din spatele ei.
- **Q16.** De ce are nevoie un server de SO_REUSEADDR în timpul dezvoltării? Este sigur să îl folosiți în producție?

Aceleași programe în Python

Modulul `socket` din Python este un strat subțire peste aceleași apeluri de sistem; `sendall` este `send_all`. Arhiva conține `python/echo_server.py` și `python/echo_client.py`, care rulează neschimbate pe Linux și pe Windows. Comparați-le rând cu rând cu versiunile în C:

```
"""echo_server.py - iterative TCP echo server (Linux and Windows). Usage: python
echo_server.py PORT"""
import socket, sys

port = int(sys.argv[1])
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as ls:
    ls.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    ls.bind(("127.0.0.1", port))
    ls.listen(8)
    print(f"listening on 127.0.0.1:{port}", flush=True)
    while True:
        conn, (ip, cport) = ls.accept()          # a NEW socket for this client
        print(f"connection from {ip}:{cport}", flush=True)
        with conn:
            total = 0
            while data := conn.recv(65536):      # b"" means the client closed
                print(f"  recv returned {len(data)} bytes", flush=True)
                total += len(data)
                conn.sendall(data)               # sendall = send_all in C
            print(f"client closed after {total} bytes", flush=True)
```

Dacă ceva nu funcționează

Simptom	Cauza probabilă și ce puteți încerca
`bind: Address already in use`	Alt program folosește portul sau o conexiune veche este în TIME_WAIT. Alegeți alt port sau așteptați; verificați cu `ss -tlnp`.
Clientul se blochează după ce trimite	Serverul citește cu un `recv_all` de dimensiune fixă și așteaptă mai mulți octeți decât s-au trimis, sau niciuna dintre părți nu închide vreodată.
Caractere fără sens după textul ecoului	O memorie tampon a fost afișată cu `%s` fără un `\0` final. Afișați în schimb cu `fwrite(buf, 1, n, stdout)`.
Merge pe localhost, dar nu între mașini	Serverul s-a legat de 127.0.0.1 sau un firewall blochează portul (`sudo ufw status`).
`getaddrinfo: Name or service not known`	Greșeală de scriere în numele gazdei sau lipsă DNS. Încercați o adresă IP.

PARTEA D · SĂPTĂMÂNA 3

Laboratorul 3 (Windows): Socket-uri I — un client și un server TCP cu Winsock

Fișă de laborator de două ore (2 × 50 de minute), însoțind Cursul 3

Windows implementează interfața Berkeley sockets ca **Winsock 2**. Conceptele și majoritatea apelurilor sunt aceleași ca pe Linux; diferențele sunt puține, dar fiecare strică un program care o ignoră. Această fișă urmează fișa de Linux sarcină cu sarcină, cu versiunile Winsock ale programelor. Ambele programe Windows au fost compilate fără avertismente cu MinGW-w64. Codul sursă și comentariile din el sunt lăsate în engleză, ca în fișierele din arhivă.

La sfârșitul laboratorului ar trebui să puteți

- să construiți programe Winsock cu MinGW-w64 sau cu Visual Studio;
- să enumerați diferențele dintre Winsock și socket-urile POSIX și să o tratați pe fiecare;
- să rulați un client și un server TCP și să le observați socket-urile cu PowerShell;
- să arătați, cu măsurători, că un flux de octeți nu are granițe între mesaje;
- să citiți codurile de eroare Winsock;
- să rulați aceleași programe Python pe Windows și pe Linux.

Înainte de a începe

- **Un compilator C.** Fie **MinGW-w64** (de exemplu din MSYS2: `pacman -S mingw-w64-ucrt-x86_64-gcc`), fie **Visual Studio Build Tools** cu sarcina de lucru (*workload*) „Desktop development with C++”; în al doilea caz folosiți *Developer PowerShell for VS*.
- **Construirea.** MinGW: `gcc echo_server_win.c -o echo_server.exe -lws2_32`. Visual Studio: `cl echo_server_win.c ws2_32.lib`. Biblioteca **ws2_32** trebuie legată întotdeauna.
- **Fișiere.** `netio_win.h`, `echo_client_win.c`, `echo_server_win.c` și dosarul `python/`.
- *Paravan de protecție (*firewall*). Serverul se leagă doar de 127.0.0.1, astfel încât Windows Firewall nu vă întrerupe. Legarea de toate adresele* `INADDR_ANY`) face ca Windows să ceară permisiune la prima rulare.
- **Ce predați.** Sursele și răspunsurile la **Q1–Q16**.

Ce este diferit în Winsock

	Linux (POSIX)	Windows (Winsock 2)
Inițializare	niciuna	`WSAStartup(MAKEWORD(2,2), &wsa)` înaintea oricărui apel; `WSACleanup()` la sfârșit
Antete și bibliotecă	`<sys/socket.h>`, `<netinet/in.h>`, `<arpa/inet.h>`, `<netdb.h>`	`<winsock2.h>`, `<ws2tcpip.h>`; se leagă **ws2_32**
Tipul socket-ului	`int` (un descriptor de fișier)	`SOCKET` (un identificator — *handle* — fără semn)
Valoarea de eșec	`-1`	`INVALID_SOCKET` (socket, accept) sau `SOCKET_ERROR` (celelalte)
Închiderea	`close(fd)`	`closesocket(s)` — `close` nu funcționează pe socket-uri

	Linux (POSIX)	Windows (Winsock 2)
Codul de eroare	`errno`, `perror`	`WSAGetLastError()`
Citire și scriere	`read`/`write` sau `recv`/`send`	doar `recv`/`send`; lungimile sunt `int`
Erori frecvente	ECONNREFUSED, EADDRINUSE, ECONNRESET	WSAECONNREFUSED (10061), WSAEADDRINUSE (10048), WSAECONNRESET (10054)
Conductă întreruptă	semnalul SIGPIPE	niciun semnal; `send` eșuează cu WSAECONNRESET sau WSAECONNABORTED

Plan

Pas	Activitate	Timp
1	Încălzire: un socket de ascultare văzut din PowerShell	10 min
2	Construiți și rulați clientul de ecou	15 min
3	Construiți și rulați serverul de ecou	20 min
4	Socket-urile văzute de Windows	5 min
	Pauză	10 min
5	Un flux nu este un șir de mesaje	15 min
6	Încadrare pe linii	10 min
7	Încadrare cu prefix de lungime	15 min
8	Stricăm lucrurile intenționat	10 min

Sarcina 1 — Un socket de ascultare văzut din PowerShell

Windows nu are `nc`, așa că porniți serverul Python din arhivă și priviți-l:

```
python python\echo_server.py 5000                                    # window A
Get-NetTCPConnection -LocalPort 5000                                  # window B
Test-NetConnection 127.0.0.1 -Port 5000
Get-NetTCPConnection -LocalPort 5000 | Format-Table LocalAddress, LocalPort,
RemoteAddress, RemotePort, State, OwningProcess
```

- **Q1.** Ce stări vedeți înainte și după `Test-NetConnection`? Scrieți cvadruplul (*4-tuple*) conexiunii pe care a făcut-o. Cine a ales portul clientului?
- **Q2.** `Test-NetConnection` se conectează și închide imediat. Ce afișează fereastra serverului și ce mesaj TCP i-a spus serverului că clientul a închis?

Sarcina 2 — Clientul de ecou

Funcțiile ajutătoare sunt aceleași ca pe Linux, cu tipurile Winsock:

```
/* netio_win.h - helpers for TCP byte streams on Windows (Winsock 2) - Computer Networks,
Lab 3 */
#ifndef NETIO_WIN_H
#define NETIO_WIN_H
#include <winsock2.h>
#include <ws2tcpip.h>
#include <stdint.h>
```

```
/* Write exactly len bytes. Returns 0 on success, -1 on error. */
static inline int send_all(SOCKET s, const void *buf, size_t len) {
    const char *p = buf;
    while (len > 0) {
        int n = send(s, p, (int)len, 0);
        if (n == SOCKET_ERROR) return -1;
        p += n; len -= (size_t)n;
    }
    return 0;
}

/* Read exactly len bytes. Returns 1 on success, 0 if the peer closed first, -1 on error.
*/
static inline int recv_all(SOCKET s, void *buf, size_t len) {
    char *p = buf;
    while (len > 0) {
        int n = recv(s, p, (int)len, 0);
        if (n == SOCKET_ERROR) return -1;
        if (n == 0) return 0;
        p += n; len -= (size_t)n;
    }
    return 1;
}

static inline int send_msg(SOCKET s, const void *buf, uint32_t len) {
    uint32_t netlen = htonl(len);
    if (send_all(s, &netlen, sizeof netlen) < 0) return -1;
    return send_all(s, buf, len);
}

static inline long recv_msg(SOCKET s, void *buf, uint32_t cap) {
    uint32_t netlen;
    int r = recv_all(s, &netlen, sizeof netlen);
    if (r <= 0) return r;
    uint32_t len = ntohl(netlen);
    if (len > cap) return -1;
    r = recv_all(s, buf, len);
    return r <= 0 ? -1 : (long)len;
}
#endif

/* echo_client_win.c - Winsock version of echo_client.c.
   Build:  gcc echo_client_win.c -o echo_client.exe -lws2_32     (MinGW-w64)
      or:  cl echo_client_win.c ws2_32.lib                        (Visual Studio)
   Usage:  echo_client.exe HOST PORT */
#include <stdio.h>
#include <string.h>
#include "netio_win.h"
#ifdef _MSC_VER
#pragma comment(lib, "ws2_32.lib")
#endif

int main(int argc, char *argv[]) {
    if (argc != 3) { fprintf(stderr, "usage: %s HOST PORT\n", argv[0]); return 1; }
    WSADATA wsa;
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) { fprintf(stderr, "WSAStartup failed\n");
return 1; }

    struct addrinfo hints = {0}, *res, *ai;
    hints.ai_family = AF_UNSPEC; hints.ai_socktype = SOCK_STREAM;
    int err = getaddrinfo(argv[1], argv[2], &hints, &res);
    if (err) { fprintf(stderr, "getaddrinfo failed: %d\n", err); WSACleanup(); return
1; }

    SOCKET s = INVALID_SOCKET;
    for (ai = res; ai != NULL; ai = ai->ai_next) {
```

```
        s = socket(ai->ai_family, ai->ai_socktype, ai->ai_protocol);
        if (s == INVALID_SOCKET) continue;
        if (connect(s, ai->ai_addr, (int)ai->ai_addrlen) == 0) break;
        closesocket(s); s = INVALID_SOCKET;
    }
    freeaddrinfo(res);
    if (s == INVALID_SOCKET) { fprintf(stderr, "connect failed: error %d\n",
WSAGetLastError()); WSACleanup(); return 1; }

    char line[1024], reply[1024];
    while (fgets(line, sizeof line, stdin) != NULL) {
        size_t len = strlen(line);
        if (send_all(s, line, len) < 0) { fprintf(stderr, "send failed: %d\n",
WSAGetLastError()); break; }
        int r = recv_all(s, reply, len);
        if (r <= 0) { fprintf(stderr, r == 0 ? "server closed the connection\n" : "recv
failed\n"); break; }
        fwrite(reply, 1, len, stdout);
    }
    closesocket(s);
    WSACleanup();
    return 0;
}

gcc echo_client_win.c -o echo_client.exe -lws2_32
.\echo_client.exe localhost 5000
```

- **Q3.** Găsiți în cod fiecare diferență Winsock din tabelul de mai sus. Care dintre ele, uitată, ar face programul să eșueze imediat?
- **Q4.** Rulați clientul când nimic nu ascultă pe portul 5000. Ce număr de eroare afișează? Căutați-l (`net helpmsg` nu acoperă Winsock; căutați „Windows Sockets error codes”). De ce durează eșecul aproximativ două secunde pe Windows, dar este imediat pe Linux?

Sarcina 3 — Serverul de ecou

```
/* echo_server_win.c - Winsock version of echo_server.c (iterative, one client at a
time).
   Build:  gcc echo_server_win.c -o echo_server.exe -lws2_32
   Usage:  echo_server.exe PORT */
#include <stdio.h>
#include <stdlib.h>
#include "netio_win.h"
#ifdef _MSC_VER
#pragma comment(lib, "ws2_32.lib")
#endif

int main(int argc, char *argv[]) {
    if (argc != 2) { fprintf(stderr, "usage: %s PORT\n", argv[0]); return 1; }
    setvbuf(stdout, NULL, _IONBF, 0);
    WSADATA wsa;
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) { fprintf(stderr, "WSAStartup failed\n");
return 1; }

    SOCKET ls = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (ls == INVALID_SOCKET) { fprintf(stderr, "socket: %d\n", WSAGetLastError());
return 1; }

    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    /* Loopback only: avoids the Windows Firewall prompt. Use INADDR_ANY to accept other
machines. */
    addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    addr.sin_port = htons((u_short)atoi(argv[1]));
```

```
    if (bind(ls, (struct sockaddr *)&addr, sizeof addr) == SOCKET_ERROR)
{ fprintf(stderr, "bind: %d\n", WSAGetLastError()); return 1; }
    if (listen(ls, 8) == SOCKET_ERROR) { fprintf(stderr, "listen: %d\n",
WSAGetLastError()); return 1; }
    printf("listening on 127.0.0.1:%s\n", argv[1]);

    for (;;) {
        struct sockaddr_in peer; int plen = sizeof peer;
        SOCKET cs = accept(ls, (struct sockaddr *)&peer, &plen);
        if (cs == INVALID_SOCKET) { fprintf(stderr, "accept: %d\n", WSAGetLastError());
continue; }
        char ip[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &peer.sin_addr, ip, sizeof ip);
        printf("connection from %s:%u\n", ip, ntohs(peer.sin_port));

        char buf[65536]; int n; long total = 0;
        while ((n = recv(cs, buf, sizeof buf, 0)) > 0) {
            printf("  recv returned %d bytes\n", n);
            total += n;
            if (send_all(cs, buf, (size_t)n) < 0) break;
        }
        printf("client closed after %ld bytes\n", total);
        closesocket(cs);
    }
}
```

```
gcc echo_server_win.c -o echo_server.exe -lws2_32
.\echo_server.exe 5000                    # window A
.\echo_client.exe 127.0.0.1 5000          # window B
.\echo_client.exe 127.0.0.1 5000          # window C, while B is still connected
```

- **Q5.** Asociați apelurile serverului cu Figura 2 din Cursul 3. Ce afișează serverul pentru fiecare client?
- **Q6.** De ce nu primește clientul din fereastra C niciun ecou până nu iese B? Este între timp conexiunea lui TCP stabilită (`Get-NetTCPConnection -LocalPort 5000`)?
- **Q7.** Înlocuiți `htons((u_short)atoi(argv[1]))` cu `(u_short)atoi(argv[1])`, reconstruiți și porniți serverul pe portul 5000. Pe ce port ascultă de fapt? Explicați.

Sarcina 4 — Socket-urile văzute de Windows

```
Get-NetTCPConnection -LocalPort 5000 | Select-Object State, RemotePort, OwningProcess
Get-Process -Id (Get-NetTCPConnection -LocalPort 5000 -State Listen).OwningProcess
netstat -ano | Select-String 5000
```

- **Q8.** Cu un client conectat, câte socket-uri aparțin procesului server și în ce stări? Ce program deține celălalt capăt?

Sarcina 5 — Un flux nu este un șir de mesaje

Trimiteți 300 000 de octeți dintr-un **singur** apel, din PowerShell, și urmăriți dimensiunile `recv` afișate de server:

```
$c = New-Object System.Net.Sockets.TcpClient('127.0.0.1', 5000)
$s = $c.GetStream()
$data = [byte[]]::new(300000)
$s.Write($data, 0, $data.Length)          # one write of 300 000 bytes
$c.Client.Shutdown('Send')                # like shutdown(SHUT_WR)
$buf = [byte[]]::new(65536); while ($s.Read($buf, 0, $buf.Length) -gt 0) { }
$c.Close()
```

- **Q9.** O singură scriere — câte apeluri `recv` pe server? Repetați de trei ori și cu 10 și 3 000 000 de octeți. Sunt dimensiunile aceleași de fiecare dată?

- **Q10.** Schimbați `sizeof buf` din apelul `recv` al serverului în 7 și tastați `hello world` în clientul C. Ce afișează serverul? De ce primește clientul tot ecoul întreg?

Sarcina 6 — Încadrare pe linii

Portați în Winsock bucla de tratare a liniilor din `upper_server.c` de pe Linux (arătată în fișa de Linux, Sarcina 6), folosind tabelul diferențelor. Testați-o cu codul PowerShell din Sarcina 5, scriind octeții lui `'abc' + [char]10 + 'def' + [char]10 + 'ghi'` în locul celor 300 000 de zerouri: folosiți `$data = [Text.Encoding]::ASCII.GetBytes(...)`.

- **Q11.** De ce nu primește niciodată răspuns ultima linie, incompletă?
- **Q12.** Ce ar trebui să facă serverul cu o linie mai lungă decât memoria lui tampon?

Sarcina 7 — Încadrare cu prefix de lungime

`netio_win.h` conține și `send_msg` și `recv_msg`. Scrieți `msg_server_win.c`, care răspunde `OK <n>` la fiecare mesaj de *n* octeți (modelul este `msg_server.c` de pe Linux), și un client care trimite mai multe mesaje înainte să citească răspunsurile.

- **Q13.** Cum găsește serverul sfârșitul fiecărui mesaj chiar și atunci când mai multe sosesc într-un singur `recv`? De ce `htonl`?
- **Q14.** Trimiteți un fișier ca un singur mesaj și comparați lungimea raportată cu `(Get-Item file).Length`.

Sarcina 8 — Stricăm lucrurile intenționat

Experiment	Ce faceți	Ce observați pe Windows
Conexiune refuzată	Vă conectați la un port pe care nu ascultă nimic.	Eroarea 10061 (WSAECONNREFUSED), după aproximativ 2 s de reîncercări.
Adresă deja folosită	Porniți două servere pe același port.	`bind: 10048` (WSAEADDRINUSE).
Cealaltă parte a închis	Închideți fereastra clientului cât timp serverul așteaptă în `recv`.	`recv` întoarce 0 (închidere normală) sau eșuează cu 10054 (WSAECONNRESET) dacă procesul a fost oprit forțat.
Scriere într-o conexiune închisă	Trimiteți după ce serverul a închis.	Niciun semnal, ca pe Linux; `send` eșuează cu 10053 sau 10054.

- **Q15.** Copiați fiecare număr de eroare și explicați mecanismul TCP din spatele lui.
- **Q16.** Windows permite de obicei unui server repornit să-și lege imediat din nou portul, acolo unde Linux are nevoie de SO_REUSEADDR. Pe Windows, SO_REUSEADDR are o semnificație diferită și periculoasă: permite unui al doilea program să se lege de un port deja folosit. Găsiți opțiunea Winsock SO_EXCLUSIVEADDRUSE în documentație și explicați ce împiedică.

Aceleași programe în Python

Fișierele din `python/` rulează neschimbate pe Windows: `python python\echo_server.py 5000` și `python python\echo_client.py 127.0.0.1 5000`. Python ascunde `WSAStartup` și diferențele de tipuri; comportamentul fluxului — și nevoia de `recv_all` — sunt exact aceleași.

```
"""echo_client.py - send lines from standard input, print the echo. Usage: python
echo_client.py HOST PORT"""
import socket, sys
```

```
def recv_all(sock, n):
    """Read exactly n bytes, or raise if the peer closes first."""
    chunks = []
    while n > 0:
        data = sock.recv(n)
        if not data:
            raise ConnectionError("server closed the connection")
        chunks.append(data)
        n -= len(data)
    return b"".join(chunks)

with socket.create_connection((sys.argv[1], int(sys.argv[2]))) as s:
    for line in sys.stdin:
        data = line.encode()
        s.sendall(data)
        print(recv_all(s, len(data)).decode(), end="")
```

Dacă ceva nu funcționează

Simptom	Cauza probabilă și ce puteți încerca
`undefined reference to __imp_socket` (MinGW)	Biblioteca Winsock nu a fost legată: adăugați `-lws2_32` la sfârșitul comenzii gcc.
Fiecare apel eșuează cu eroarea 10093	WSANOTINITIALISED: `WSAStartup` nu a fost apelat mai întâi.
Paravanul de protecție cere permisiune	Serverul se leagă de toate adresele. Pentru laborator, legați-l de 127.0.0.1 (cum face codul furnizat).
`gcc` nu este recunoscut	Deschideți terminalul MSYS2 UCRT64 sau adăugați dosarul lui `bin` în PATH.
Python: `ConnectionRefusedError: [WinError 10061]`	Nimic nu ascultă pe acel port; porniți mai întâi serverul.

Săptămâna viitoare

Cursul 4 continuă cu Web-ul: cookie-uri, memorare în cache (*caching*), HTTP/2, HTTP/3 și rețele de distribuire a conținutului (*content distribution networks*, CDN). Laboratorul 4 trece la socket-uri UDP și la ordinea octeților (*byte order*).